

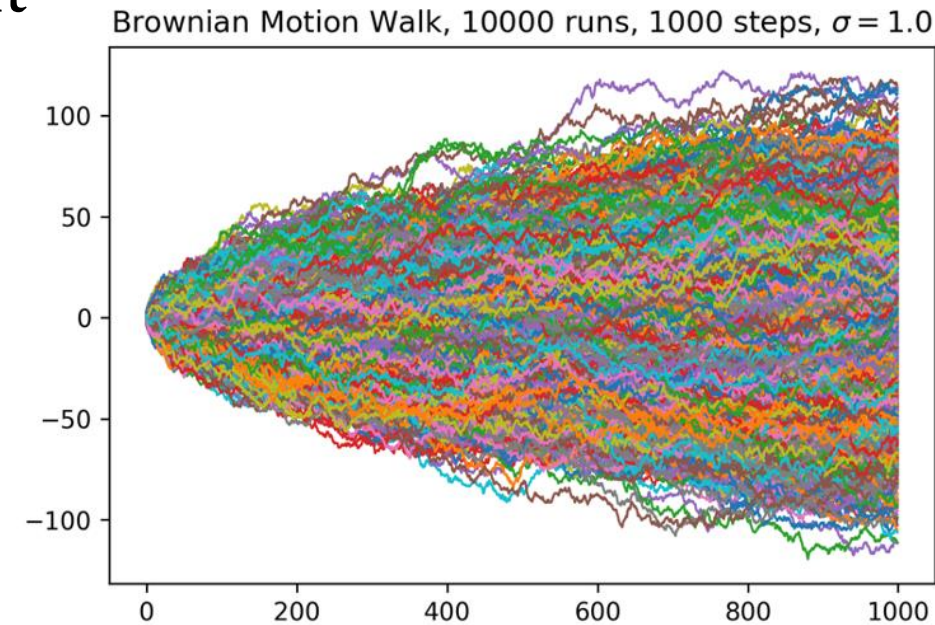
NN-GP and neural tangent kernels (NTK)

stats403_deep_learning
spring_2025
lecture_11

11.1 neural networks as Gaussian Processes (NN-GP)

Gaussian processes

- a **Gaussian process (GP)** is a probability distribution over functions $y(\mathbf{x})$ such that the set of values of $y(\mathbf{x})$ evaluated at an arbitrary set of points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$ jointly have a Gaussian distribution
- in high-dim, also called a **Gaussian random field**
- A GP may or may not be stationary
- example: Brownian motion



Gaussian processes

- fact: a Gaussian process is **completely** determined by
 - expectation $\mathbb{E}[y(\mathbf{x})] = \mu(\mathbf{x})$
 - covariance $\text{Cov}(y(\mathbf{x}), y(\mathbf{x}')) = \mathbf{K}(\mathbf{x}, \mathbf{x}')$
- we can denote such a Gaussian process as $\mathcal{GP}(\mu, \mathbf{K})$

Gaussian processes

- in most applications, we will not have any prior knowledge about the mean of $y(\mathbf{x})$ and so by symmetry we take it to be zero
- the specification of the Gaussian process is then completed by giving the covariance of $y(\mathbf{x})$ evaluated at any two values of $\mathbf{x}$
- the covariance is given by $\mathcal{GP}(0, K(\cdot, \cdot))$

$$\mathbb{E} [y(\mathbf{x}_n)y(\mathbf{x}_m)] = k(\mathbf{x}_n, \mathbf{x}_m)$$

Gaussian processes

- e.g., in GP-regression, we can start with a nonlinear mapping ϕ :

$$\left. \begin{array}{l} y(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x}) \\ p(\mathbf{w}) = \mathcal{N}(\mathbf{w} | \mathbf{0}, \alpha^{-1} \mathbf{I}) \end{array} \right\} \longrightarrow K_{nm} = k(\mathbf{x}_n, \mathbf{x}_m) = \frac{1}{\alpha} \phi(\mathbf{x}_n)^T \phi(\mathbf{x}_m)$$

- we can also model the kernel directly

Gaussian processes

- Constant : $K_C(x, x') = C$

- Linear: $K_L(x, x') = x^T x'$

- white Gaussian noise:

$$K_{GN}(x, x') = \sigma^2 \delta_{x, x'}$$

- Squared exponential:

$$K_{SE}(x, x') = \exp\left(-\frac{|d|^2}{2\ell^2}\right)$$

- Ornstein–Uhlenbeck: $K_{OU}(x, x') = \exp\left(-\frac{|d|}{\ell}\right)$

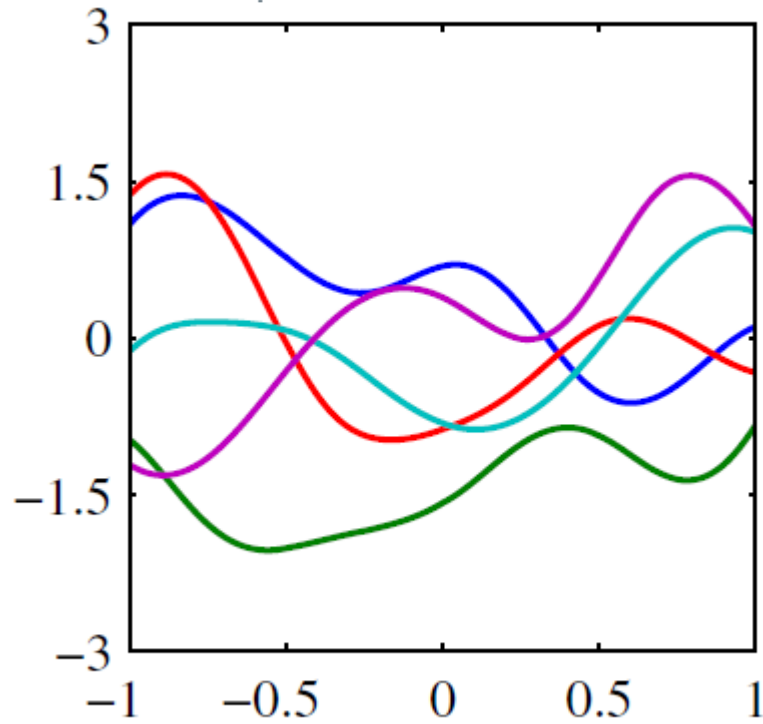
- Matérn: $K_{\text{Matern}}(x, x') = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}|d|}{\ell}\right)^\nu K_\nu\left(\frac{\sqrt{2\nu}|d|}{\ell}\right)$

- Periodic: $K_P(x, x') = \exp\left(-\frac{2 \sin^2\left(\frac{d}{2}\right)}{\ell^2}\right)$

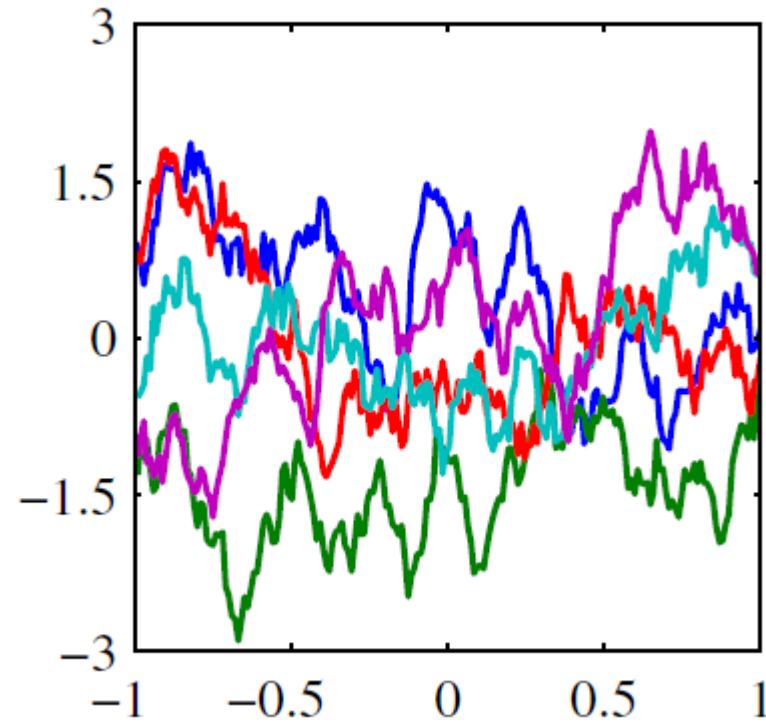
- Rational quadratic: $K_{RQ}(x, x') = (1 + |d|^2)^{-\alpha}, \quad \alpha \geq 0$

$$(\quad d = x - x' \quad)$$

Gaussian processes



squared exponential



Ornstein-Uhlenbeck

GP for regression

- Consider $t_n = y_n + \epsilon_n$ with $p(t_n|y_n) = \mathcal{N}(t_n|y_n, \beta^{-1})$
- assume the random variable y follows the GP described above

$$\mathcal{GP}(0, K(\cdot, \cdot))$$

- the joint distribution of the target values $\mathbf{t} = (t_1, \dots, t_N)^T$ conditioned on $\mathbf{y} = (y_1, \dots, y_N)^T$

$$p(\mathbf{t}|\mathbf{y}) = \mathcal{N}(\mathbf{t}|\mathbf{y}, \beta^{-1}\mathbf{I}_N)$$

- the prior is, according to the GP,

$$p(\mathbf{y}) = \mathcal{N}(\mathbf{y}|\mathbf{0}, \mathbf{K})$$

GP for regression

- by a fact of Gaussian, the marginal distribution of $\mathbf{t}$ is

$$p(\mathbf{t}) = \int p(\mathbf{t}|\mathbf{y})p(\mathbf{y}) \, d\mathbf{y} = \mathcal{N}(\mathbf{t}|\mathbf{0}, \mathbf{C}_N)$$

where the (n, m) -th entry of $\mathbf{C}_N$ is given by

$$C_N(\mathbf{x}_n, \mathbf{x}_m) = k(\mathbf{x}_n, \mathbf{x}_m) + \beta^{-1}\delta_{nm}$$

- this also holds if we replace N by $N + 1$

GP for regression

$$p(\mathbf{t}) = \int p(\mathbf{t}|\mathbf{y})p(\mathbf{y}) d\mathbf{y} = \mathcal{N}(\mathbf{t}|\mathbf{0}, \mathbf{C})$$

- consider a test point $\mathbf{x}_{N+1}$
- want to find $p(t_{N+1}|\mathbf{t})$
- first consider the joint distribution $p(\mathbf{t}_{N+1})$ where
$$\mathbf{t}_{N+1} = (t_1, \dots, t_N, t_{N+1})^T$$

$$p(\mathbf{t}_{N+1}) = \mathcal{N}(\mathbf{t}_{N+1}|\mathbf{0}, \mathbf{C}_{N+1})$$

$$\mathbf{C}_{N+1} = \begin{pmatrix} \mathbf{C}_N & \mathbf{k} \\ \mathbf{k}^T & c \end{pmatrix}$$

$\mathbf{k}$ has elements $k(\mathbf{x}_n, \mathbf{x}_{N+1})$ for $n = 1, \dots, N$

$c = k(\mathbf{x}_{N+1}, \mathbf{x}_{N+1}) + \beta^{-1}$

derived according to “this also holds if we replace N by $N + 1$ ”

GP for regression

- then from analysis of Gaussian, $p(t_{N+1}|\mathbf{t})$ is also Gaussian with mean and variance given by

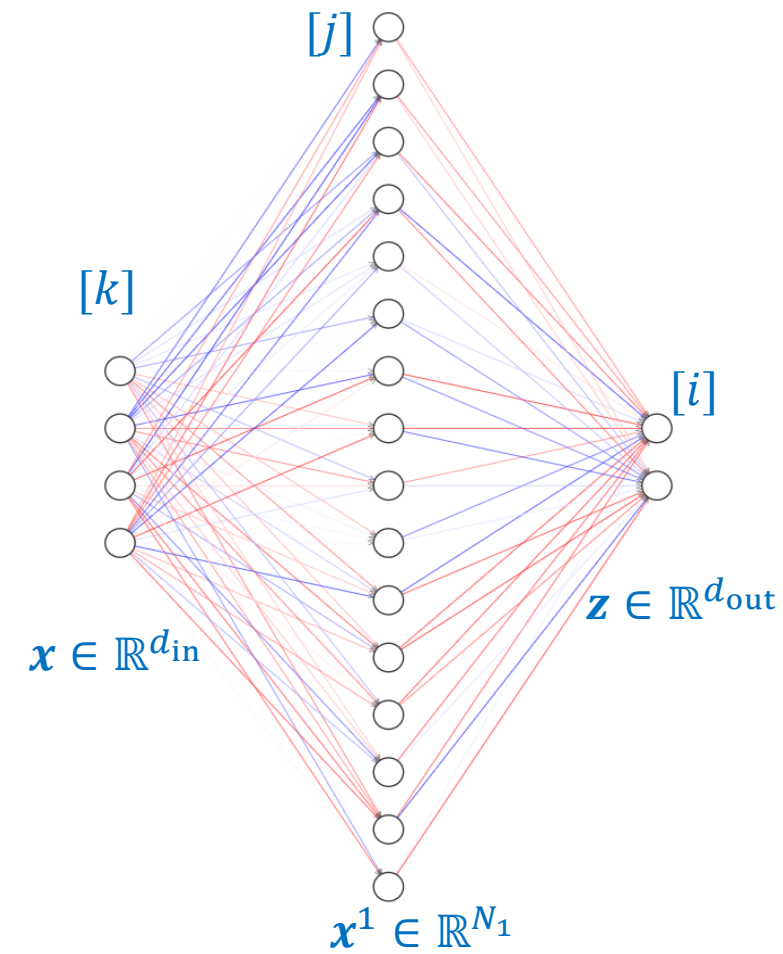
$$\begin{aligned}m(\mathbf{x}_{N+1}) &= \mathbf{k}^T \mathbf{C}_N^{-1} \mathbf{t} \\ \sigma^2(\mathbf{x}_{N+1}) &= c - \mathbf{k}^T \mathbf{C}_N^{-1} \mathbf{k}\end{aligned}$$

NN as GP

- First, let's consider 1(hidden)-layer network

$$x_j^1(x) = \phi\left(b_j^0 + \sum_{k=1}^{d_{in}} W_{jk}^0 x_k\right)$$

$$z_i^1(x) = b_i^1 + \sum_{j=1}^{N_1} W_{ij}^1 x_j^1(x)$$



- If we take W_{ij}^l, b_i^l to be zero mean and variances are σ_w^2/N_l and σ_b^2 , respectively. What can we say for $z_i^1(x)$?

NN as GP

$$\begin{aligned} x_j^1(x) &= \phi\left(b_j^0 + \sum_{k=1}^{d_{in}} W_{jk}^0 x_k\right) \\ z_i^1(x) &= b_i^1 + \sum_{j=1}^{N_1} W_{ij}^1 x_j^1(x) \end{aligned} \quad \left\{ \begin{array}{l} W_{ij}^l \sim \mathcal{N}(0, \sigma_w^2/N_l) \\ b_i^l \sim \mathcal{N}(0, \sigma_b^2) \end{array} \right.$$

- let's take $N_1 \rightarrow \infty$

Lindeberg–Lévy CLT. Suppose $\{X_1, \dots, X_n\}$ is a sequence of **i.i.d.** random variables with $\mathbb{E}[X_i] = \mu$ and $\text{Var}[X_i] = \sigma^2 < \infty$. Then as n approaches infinity, the random variables $\sqrt{n}(\bar{X}_n - \mu)$ **converge in distribution** to a **normal** $\mathcal{N}(0, \sigma^2)$:^[4]

$$\sqrt{n}(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2).$$

NN as GP

$$\begin{aligned} x_j^1(x) &= \phi\left(b_j^0 + \sum_{k=1}^{d_{in}} W_{jk}^0 x_k\right) \\ z_i^1(x) &= b_i^1 + \sum_{j=1}^{N_1} W_{ij}^1 x_j^1(x) \end{aligned} \quad \left\{ \begin{array}{l} W_{ij}^l \sim \mathcal{N}(0, \sigma_w^2/N_l) \\ b_i^l \sim \mathcal{N}(0, \sigma_b^2) \end{array} \right.$$

- let's take $N_1 \rightarrow \infty$
- then $z_i^1 \sim \mathcal{GP}(\mu^1, K^1)$ where

$$\mu^1(x) = \mathbb{E}[z_i^1(x)] = 0$$

$$K^1(x, x') \equiv \mathbb{E}[z_i^1(x)z_i^1(x')] = \sigma_b^2 + \sigma_w^2 \mathbb{E}[x_i^1(x)x_i^1(x')] \equiv \sigma_b^2 + \sigma_w^2 C(x, x')$$

NN as GP

$$x_j^1(x) = \phi\left(b_j^0 + \sum_{k=1}^{d_{in}} W_{jk}^0 x_k\right) \quad \begin{cases} W_{ij}^l \sim \mathcal{N}(0, \sigma_w^2/N_l) \\ b_i^l \sim \mathcal{N}(0, \sigma_b^2) \end{cases}$$

- now consider a deep NN with L layers

$$z_i^l(x) = b_i^l + \sum_{j=1}^{N_l} W_{ij}^l x_j^l(x) \quad 1 \leq l \leq L$$

$$x_j^l(x) = \phi(z_j^{l-1}(x)) \quad 2 \leq l \leq L-1$$

- as $N_l \rightarrow \infty$, $z_i^l \sim \mathcal{GP}(0, K^l)$, where

$$K^l(x, x') \equiv \mathbb{E}[z_i^l(x) z_i^l(x')] = \sigma_b^2 + \sigma_w^2 \mathbb{E}_{z_i^{l-1} \sim \mathcal{GP}(0, K^{l-1})} [\phi(z_i^{l-1}(x)) \phi(z_i^{l-1}(x'))]$$

- if we let the number of neurons of each layer to go to infinity successively, the output of NN converges to a GP

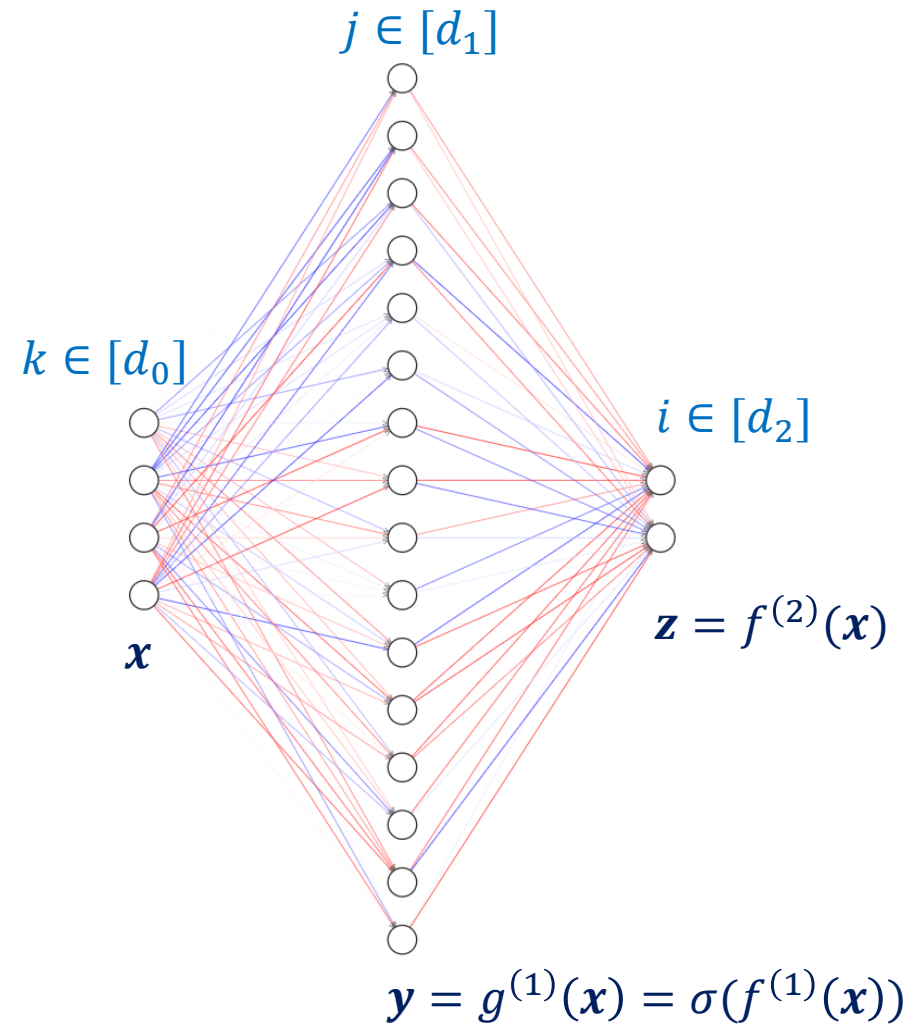
NN as GP

- a simplified inductive relation can be written as

$$K^l(x, x') = \sigma_b^2 + \sigma_w^2 F_\phi\left(K^{l-1}(x, x'), K^{l-1}(x, x), K^{l-1}(x', x')\right)$$

use the NN-GP

- in GP regression, we take the prior $p(\mathbf{y})$ to be a GP
- we can take it to be the NN-GP



11.2 motivation of neural tangent kernel

why does the gradient have to do with kernel methods?

- consider a simple kernel regression model $f(\boldsymbol{\theta}, \boldsymbol{x}) = \boldsymbol{\theta}^T \boldsymbol{\phi}(\boldsymbol{x})$
- what is the kernel function K ?

11.2 neural tangent

gradient descent

$$\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n \subset \mathbb{R}^d \times \mathbb{R}$$

$$\ell(\boldsymbol{\theta}) = \frac{1}{2} \sum_{i=1}^n (f(\boldsymbol{\theta}, \boldsymbol{x}_i) - y_i)^2$$

$$\theta(t+1) = \theta(t) - \eta \nabla_{\theta} \ell(\theta(t))$$

gradient descent with very small LR

$$\frac{d\boldsymbol{\theta}(t)}{dt} = -\nabla \ell(\boldsymbol{\theta}(t))$$

- let

$$\mathbf{u}(t) = (f(\boldsymbol{\theta}(t), \mathbf{x}_i))_{i \in [n]} \in \mathbb{R}^n$$

be the network output

- then $\mathbf{u}(t)$ follows the following evolution

$$\frac{d\mathbf{u}(t)}{dt} = -\underbrace{\mathbf{H}(t)} \cdot (\mathbf{u}(t) - \mathbf{y})$$

where $\mathbf{H}(t)$'s (i, j) -th entry is $\left\langle \frac{\partial f(\boldsymbol{\theta}(t), \mathbf{x}_i)}{\partial \boldsymbol{\theta}}, \frac{\partial f(\boldsymbol{\theta}(t), \mathbf{x}_j)}{\partial \boldsymbol{\theta}} \right\rangle$

why?

gradient descent with very small LR

To see why, first look at

$$\frac{d\theta(t)}{dt} = -\nabla_{\theta} l(\theta(t))$$

$$\left[\text{where } l(\theta(t)) = \frac{1}{2} \sum_{j=1}^n (f(\theta(t), x_j) - y_j)^2 \right]$$

$$= - \sum_{j=1}^n (f(\theta(t), x_j) - y_j) \frac{\partial f(\theta(t), x_j)}{\partial \theta}$$

gradient descent with very small LR

$$\begin{aligned}\text{Also, } \frac{df(\theta(t), x_i)}{dt} &= \left\langle \frac{\partial f(\theta(t), x_i)}{\partial \theta}, \frac{d\theta(t)}{dt} \right\rangle \\ &= - \sum_{j=1}^n (f(\theta(t), x_j) - y_j) \underbrace{\left\langle \frac{\partial f(\theta(t), x_i)}{\partial \theta}, \frac{\partial f(\theta(t), x_j)}{\partial \theta} \right\rangle}_{H_{ij}(t)}\end{aligned}$$

$$= - \sum_{j=1}^n (f(\theta(t), x_j) - y_j) H_{ij}(t)$$

$$\text{That is, } \frac{du_i}{dt} = - \sum_{j=1}^n (f(\theta(t), x_j) - y_j) H_{ij}(t)$$

$$\text{therefore, } \frac{du(t)}{dt} = - H(t) \cdot (u(t) - y)$$

gradient descent with very small LR

- in the limit where the width of NN goes to infinity

$$\frac{d\mathbf{u}(t)}{dt} = -\mathbf{H}^* \cdot (\mathbf{u}(t) - \mathbf{y})$$


A blue arrow points from the expression $\mathbf{H}(t)$ above to the $\mathbf{H}^*$ term in the equation below.

NTK dynamics

- we can solve $\frac{d\mathbf{u}(t)}{dt} = -\mathbf{H}^* \cdot (\mathbf{u}(t) - \mathbf{y})$

Let $v(t) = u(t) - y$. Then $\frac{dv(t)}{dt} = -H^* v(t)$

Then $v(t) = e^{-t H^*} v(0)$

That is, $u(t) = y + e^{-t H^*} (u(0) - y)$

11.3 derivation of NTK

derivation of NTK

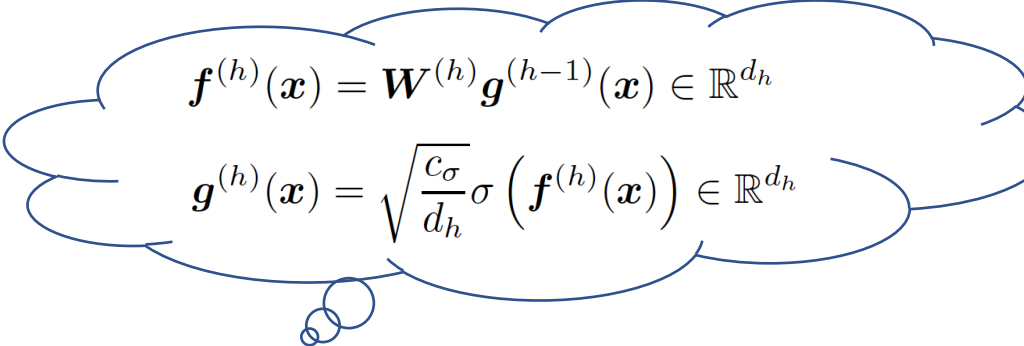
$$\boldsymbol{x} \in \mathbb{R}^d \quad \boldsymbol{g}^{(0)}(\boldsymbol{x}) = \boldsymbol{x} \quad d_0 = d \quad \text{input}$$

$$\boldsymbol{f}^{(h)}(\boldsymbol{x}) = \boldsymbol{W}^{(h)} \boldsymbol{g}^{(h-1)}(\boldsymbol{x}) \in \mathbb{R}^{d_h} \quad \text{linear transformation}$$

$$\boldsymbol{g}^{(h)}(\boldsymbol{x}) = \sqrt{\frac{c_\sigma}{d_h}} \sigma \left(\boldsymbol{f}^{(h)}(\boldsymbol{x}) \right) \in \mathbb{R}^{d_h}, \quad h = 1, 2, \dots, L, \quad \text{nonlinear activation}$$

$$\text{where } c_\sigma = \left(\mathbb{E}_{z \sim \mathcal{N}(0,1)} \left[\sigma(z)^2 \right] \right)^{-1} \quad \text{normalizing factor}$$

derivation of NTK


$$\mathbf{f}^{(h)}(\mathbf{x}) = \mathbf{W}^{(h)} \mathbf{g}^{(h-1)}(\mathbf{x}) \in \mathbb{R}^{d_h}$$

$$\mathbf{g}^{(h)}(\mathbf{x}) = \sqrt{\frac{c_\sigma}{d_h}} \sigma \left(\mathbf{f}^{(h)}(\mathbf{x}) \right) \in \mathbb{R}^{d_h}$$

$$f(\boldsymbol{\theta}, \mathbf{x}) = f^{(L+1)}(\mathbf{x}) = \mathbf{W}^{(L+1)} \cdot \mathbf{g}^{(L)}(\mathbf{x})$$

$$= \mathbf{W}^{(L+1)} \cdot \sqrt{\frac{c_\sigma}{d_L}} \sigma \left(\mathbf{W}^{(L)} \cdot \sqrt{\frac{c_\sigma}{d_{L-1}}} \sigma \left(\mathbf{W}^{(L-1)} \dots \sqrt{\frac{c_\sigma}{d_1}} \sigma \left(\mathbf{W}^{(1)} \mathbf{x} \right) \right) \right)$$

- the weights are initialized as i.i.d. $\mathcal{N}(0,1)$

derivation of NTK

- **remark:** under this notation, the NN-GP at infinite width

$$\Sigma^{(0)}(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{x}', \quad \text{covariance for initial layer}$$

$$\mathbf{\Lambda}^{(h)}(\mathbf{x}, \mathbf{x}') = \begin{pmatrix} \Sigma^{(h-1)}(\mathbf{x}, \mathbf{x}) & \Sigma^{(h-1)}(\mathbf{x}, \mathbf{x}') \\ \Sigma^{(h-1)}(\mathbf{x}', \mathbf{x}) & \Sigma^{(h-1)}(\mathbf{x}', \mathbf{x}') \end{pmatrix} \in \mathbb{R}^{2 \times 2}, \quad \text{covariance from previous layer}$$

$$\Sigma^{(h)}(\mathbf{x}, \mathbf{x}') = c_\sigma \mathbb{E}_{(u,v) \sim \mathcal{N}(\mathbf{0}, \mathbf{\Lambda}^{(h)})} [\sigma(u) \sigma(v)]. \quad \text{covariance for current layer}$$

$$\left\langle \frac{\partial f(\boldsymbol{\theta}(t), \mathbf{x}_i)}{\partial \boldsymbol{\theta}}, \frac{\partial f(\boldsymbol{\theta}(t), \mathbf{x}_j)}{\partial \boldsymbol{\theta}} \right\rangle$$

derivation of NTK

- now let's study the kernel $\mathbf{H}^*$ defined for every given pair $\mathbf{x}, \mathbf{x}'$:

$$\mathbf{H}^*(\mathbf{x}, \mathbf{x}') = \left\langle \frac{\partial f(\boldsymbol{\theta}, \mathbf{x})}{\partial \boldsymbol{\theta}}, \frac{\partial f(\boldsymbol{\theta}, \mathbf{x}')}{\partial \boldsymbol{\theta}} \right\rangle$$

$$f(\theta, x) \leftarrow \boxed{W^{(L+1)}} \leftarrow g^{(L)}(x) \leftarrow \boxed{\sqrt{\frac{C_\sigma}{d_L}} \sigma} \leftarrow f^{(L)}(x) \leftarrow \boxed{W^{(L)}} \leftarrow g^{(L-1)}(x) \leftarrow \vdots$$

• First, $f(\theta, x) = \underbrace{W^{(L+1)}}_{1 \times d_L} \underbrace{g^{(L)}(x)}_{d_L \times 1}$

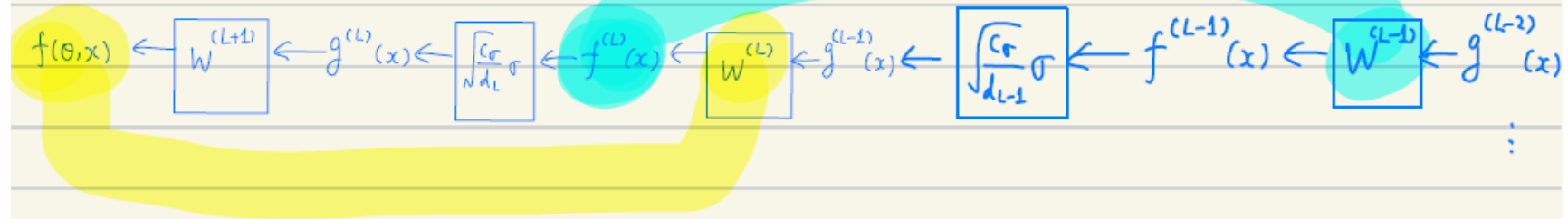
$$\frac{\partial f(\theta, x)}{\partial W^{(L+1)}} = \left(g^{(L)}(x) \right)^T$$

• Next, $f^{(L)}(x) = W^{(L+1)} \left[\frac{c_\sigma}{d_L} \sigma \left(W^{(L)} g^{(L-1)}(x) \right) \right]$

$$\frac{\partial f^{(L)}(x)}{\partial W^{(L)}} = \sum_i \underbrace{\frac{\partial f^{(L)}(x)}{\partial g_i^{(L)}(x)}}_{W_i^{(L+1)}} \cdot \underbrace{\frac{\partial g_i^{(L)}(x)}{\partial f_i^{(L)}(x)}}_{\sqrt{\frac{c_\sigma}{d_L}} \sigma'(f_i^{(L)}(x))} \cdot \underbrace{\frac{\partial f_i^{(L)}(x)}{\partial W^{(L)}}}_{\begin{bmatrix} 0 & (g^{(L-1)}(x))^T \\ 0 & \end{bmatrix} \text{ i-th row}}$$

$$b^{(L)}(x) = \boxed{\sqrt{\frac{c_\sigma}{d_L}} \underbrace{D^{(L)}(x)}_{d_L \times d_L} \underbrace{\left(W^{(L+1)} \right)^T}_{d_L \times 1} \underbrace{\left(g^{(L-1)}(x) \right)^T}_{1 \times d_L}}$$

where $D^{(L)}(x) = \text{diag}(\sigma'(f_i^{(L)}(x)))$



• Next,

$$\frac{\partial f(\theta, x)}{\partial W^{(L-1)}} = \sum_i \underbrace{\frac{\partial f(\theta, x)}{\partial g_i^{(L)}(x)} \frac{\partial g_i^{(L)}(x)}{\partial f_i^{(L)}(x)}}_{b_i^{(L)}(x)} \underbrace{\frac{\partial f_i^{(L)}(x)}{\partial W^{(L-1)}}}_{\downarrow}$$

$$\sqrt{\frac{c_\sigma}{d_{L-1}}} D^{(L-1)}(x) \left(W_i^{(L)} \right)^T \left(g^{(L-2)}(x) \right)^T$$

where $D^{(L-1)}(x) = \text{diag}(\sigma'(f_i^{(L-1)}(x)))$

$$\frac{\partial f(\theta, x)}{\partial W^{(L-1)}} = \sum_i \frac{\partial f(\theta, x)}{\partial g_i^{(L)}(x)} \frac{\partial g_i^{(L)}(x)}{\partial f_i^{(L)}(x)} \frac{\partial f_i^{(L)}(x)}{\partial W^{(L-1)}}$$

$$= \sum_i b_i^{(L)}(x) \sqrt{\frac{c_\sigma}{d_{L-1}}} D^{(L-1)}(x) \underbrace{(W_i^{(L)})^T}_{d_{L-1} \times 1} (g^{(L-2)}(x))^T$$

$$\underbrace{b^{(L-1)}(x)}_{\leftarrow} = \sqrt{\frac{c_\sigma}{d_{L-1}}} \underbrace{D^{(L-1)}(x)}_{d_{L-1} \times d_{L-1}} \underbrace{(W^{(L)})^T}_{d_{L-1} \times d_L} \underbrace{b^{(L)}(x)}_{d_L \times 1} \underbrace{(g^{(L-2)}(x))^T}_{1 \times d_L}$$

Now we see the trend!

- By induction, define recursively

$$b^{(h)}(x) = \begin{cases} 1, & h = L+1 \\ \sqrt{\frac{C_0}{d_h}} D^{(h)}(x) (W^{(h+1)})^T b^{(h+1)}(x), & h = L, L-1, \dots, 2, 1. \end{cases}$$

where $D^{(h)}(x) = \text{diag}(\sigma(f_i^{(h)}(x)))$.

Then $\frac{\partial f(\theta, x)}{\partial W^{(h)}} = b^{(h)}(x) (g^{(h-1)}(x))^T$.

$$h = 1, 2, \dots, L, L+1.$$

• Next, we need to evaluate $\left\langle \frac{\partial f(\theta, x)}{\partial \theta}, \frac{\partial f(\theta, x')}{\partial \theta} \right\rangle$.

$$\parallel$$
$$\sum_{h=1}^{L+1} \left\langle \frac{\partial f(\theta, x)}{\partial w^{(h)}}, \frac{\partial f(\theta, x')}{\partial w^{(h)}} \right\rangle$$

We write $\left\langle \frac{\partial f(\theta, x)}{\partial w^{(h)}}, \frac{\partial f(\theta, x')}{\partial w^{(h)}} \right\rangle$

$$= \left\langle b^{(h)}(x) \left(g^{(h-1)}(x) \right)^T, b^{(h)}(x') \left(g^{(h-1)}(x') \right)^T \right\rangle$$

[recall that $\langle A, B \rangle = \sum_{i,j} A_{ij} B_{ij} = \text{tr}(A^T B)$]

$$= \text{tr} \left(\underbrace{g^{(h-1)}(x) \left(b^{(h)}(x) \right)^T}_{\text{matrix}} \underbrace{b^{(h)}(x') \left(g^{(h-1)}(x') \right)^T}_{\text{matrix}} \right)$$

$$= \cancel{\text{tr}} \left((b^{(h)}(x))^T b^{(h)}(x') (g^{(h-1)}(x'))^T g^{(h-1)}(x) \right)$$

$$= \langle b^{(h)}(x), b^{(h)}(x') \rangle \underbrace{\langle g^{(h-1)}(x), g^{(h-1)}(x') \rangle}_{\text{described by NN-GP,}}$$

described by NN-GP,

$$\Sigma^{(h-1)}(x, x')$$

On the other hand, $\langle b^{(h)}(x), b^{(h)}(x') \rangle$

$$= \left\langle \sqrt{\frac{C_\sigma}{N d_h}} D^{(h)}(x) (W^{(h+1)})^T b^{(h+1)}(x), \right.$$

$$\left. \sqrt{\frac{C_\sigma}{N d_h}} D^{(h)}(x') (W^{(h+1)})^T b^{(h+1)}(x') \right\rangle$$

In general, $W^{(h+1)}$ and $b^{(h+1)}$ are correlated.

Nevertheless, at infinite width, they are independent.

$$= \frac{C\sigma}{dh} \langle D W^T b, D' W^T b' \rangle \quad \left(\begin{array}{l} \text{write the short} \\ \text{ver. due to} \\ \text{lazyness} \end{array} \right)$$

$$= \frac{C\sigma}{dh} \sum_i \left(\sigma(f_i^{(h)}(x)) \sum_j W_{ji} b_j \right) \left(\sigma(f_i^{(h)}(x')) \sum_k W_{ki} b'_k \right)$$

$$= \frac{C\sigma}{dh} \sum_i \sigma(f_i^{(h)}(x)) \sigma(f_i^{(h)}(x')) \sum_j \sum_k \underbrace{W_{ji} W_{ki}} b_j b'_k$$

"trick": replace by an
"expected version":

$$\approx \begin{cases} 0, & \text{if } j \neq k \\ 1, & \text{if } j = k \end{cases}$$

$$= \frac{C_\sigma}{d_h} \sum_i \dot{\sigma}(f_i^{(h)}(x)) \dot{\sigma}(f_i^{(h)}(x')) \sum_j b_j b_j'$$

$$= \frac{C_\sigma}{d_h} \langle D^{(h)}(x), D^{(h)}(x') \rangle \langle b^{(h+1)}(x), b^{(h+1)}(x') \rangle$$

That is, $\langle b^{(h)}(x), b^{(h)}(x') \rangle$

$$= \frac{C_\sigma}{d_h} \langle D^{(h)}(x), D^{(h)}(x') \rangle \langle b^{(h+1)}(x), b^{(h+1)}(x') \rangle$$

$$\dot{\Sigma}^{(h)}(x, x') := C_\sigma \mathbb{E}_{(u, v) \sim N(0, \Lambda^{(h)})} [\dot{\sigma}(u) \dot{\sigma}(v)]$$

- Therefore, by induction,

$$\langle b^{(h)}(x), b^{(h)}(x') \rangle = \prod_{h'=h}^L \dot{\Sigma}^{(h')}(x, x')$$

To summarize,

$$\left\langle \frac{\partial f(\theta, x)}{\partial \theta}, \frac{\partial f(\theta, x')}{\partial \theta} \right\rangle$$

$$= \sum_{h=1}^{L+1} \left\langle \frac{\partial f(\theta, x)}{\partial w^{(h)}}, \frac{\partial f(\theta, x')}{\partial w^{(h)}} \right\rangle$$

$$= \sum_{h=1}^{L+1} \langle b^{(h)}(x), b^{(h)}(x') \rangle \Sigma^{(h-1)}(x, x')$$

$$= \Sigma^{(L)}(x, x') + \sum_{h=1}^L \Sigma^{(h-1)}(x, x') \prod_{h'=h}^L \dot{\Sigma}^{(h')}(x, x')$$

NTK cookbook

- https://colab.research.google.com/github/google/neural-tangents/blob/master/notebooks/neural_tangents_cookbook.ipynb

Thank you!

Reference

- Ch 6.4.2, 6.4.7, Bishop, C., *PRML textbook*.
- Lee, J., Bahri, Y., Novak, R., Schoenholz, S. S., Pennington, J., & Sohl-Dickstein, J. (2017). Deep neural networks as gaussian processes. *ICLR*.
- Neal, R. M. (1996). *Bayesian Learning for Neural Networks*. Springer. Lecture Notes in Statistics 118.
- Arora, S., Du, S. S., Hu, W., Li, Z., Salakhutdinov, R., & Wang, R. (2019). On exact computation with an infinitely wide neural net. In *NeurIPS*.
- Jacot, A., Hongler, C., & Gabriel, F. (2018). Neural Tangent Kernel: Convergence and Generalization in Neural Networks. In *NeurIPS*.
- Novak, R., Xiao, L., Hron, J., Lee, J., Alemi, A. A., Sohl-Dickstein, J., & Schoenholz, S. S. (2019). Neural tangents: Fast and easy infinite neural networks in python. In *ICLR*.

